Sales Analytics Pipeline

Phase 3 — Development & Deployment Runbook

How to run the SQL package end-to-end on a Snowflake trial account, including the 7-day simulation

Prepared by: Jerry Wyatt

Date: August 14, 2026

Revision: v2 — reworks custom activity field resolution (CUSTOM_ACTIVITY_TYPES + CUSTOM_ACTIVITY_FIELDS) after reviewing the explanation video and GoodNotes transcript; see Section 8

Status: Draft — run and verify on your trial account, then review with SME

Builds on: Phase 1 (Architecture v3) and Phase 2 (ERD v3)

1. Purpose & What's Included

This is the Phase 3 build: a complete, runnable Snowflake SQL package implementing the Bronze -> Silver -> Gold -> Export pipeline designed in Phase 1 and Phase 2, plus a synthetic 7-day data set so the whole thing can be run and verified end-to-end without needing the real PostgreSQL source. No Spark, PySpark, Databricks, or dbt is used anywhere -- everything is native Snowflake (SQL, JavaScript UDFs, Snowflake Scripting stored procedures, Tasks, and Streams), per the project constraint.

I don't have a live connection to your Snowflake account in this session, so none of this has been executed against real Snowflake yet -- run it yourself in Snowsight (or SnowSQL) following the steps below, and treat this as a first pass to debug against your actual trial account's quirks.

File	Purpose
00_generate_sample_data.py	Generates 7 days of synthetic source JSON (leads A-G worked examples + 8 filler leads); custom fields land as top-level custom.cf_<hash> keys, resolved via a nested type/field dictionary, matching the real Close CRM shape
01_stages_and_formats.sql	Database/schema/warehouse setup, JSON file format, internal landing stage, export stage
01b_put_sample_data.sql	PUT commands (SnowSQL) to push all 7 days of sample files to the internal stage
02_bronze_ddl.sql	Bronze table DDL (LEAD_ACTIVITIES_RAW, CUSTOM_ACTIVITIES_RAW, CLOSE_CRM_USERS_RAW, LEADS_RAW)
03_bronze_copy_into.sql	BRONZE.LOAD_BRONZE_DAY(date) procedure -- COPY INTO for one day's files
04_json_repair_udf.sql	REPAIR_SINGLE_QUOTED_JSON / REPAIR_AND_PARSE_JSON UDFs + smoke test
05_silver_transient_flatten.sql	All Silver table DDL + SILVER.LOAD_SILVER_TRANSIENT(date) procedure (FLATTEN)
06_silver_merge_dedup.sql	SILVER.MERGE_SILVER_DAY(date) procedure -- MD5_HASH + MERGE upserts for every Silver table
07_gold_views.sql	The five Gold business-logic views implementing the funnel rules
08_export_unload.sql	Four report views + COPY INTO unloads to the export stage
09_orchestration_tasks_streams.sql	Streams + Task DAG for production scheduling, plus the stage-purge procedure

2. Prerequisites

- A Snowflake trial account, logged into Snowsight, with a role that can CREATE DATABASE / WAREHOUSE / TASK (ACCOUNTADMIN or SYSADMIN on a fresh trial both work).
- Python 3 locally, to run the sample data generator (no extra packages required -- stdlib only).
- Optional: SnowSQL CLI installed, for the PUT commands in 01b. If you don't want to install SnowSQL, Snowsight's "Upload Files" button on a stage works too -- see Section 4, Step 3.

3. Step 1 — Generate the Sample Data

From the sql/ folder:

```
python3 00_generate_sample_data.py
```

This creates sample_data/day_1/ through sample_data/day_7/, each with the four source JSON files. Day 1 has 9 activities; by day 7 the cumulative file has 30, across 15 leads (7 worked examples + 8 random fillers). Lead C's Strategy Call outcome is deliberately logged wrong on days 1-3 and corrected on day 4+ -- this is what exercises the MD5_HASH change-detection path in Step 5.

4. Step 2 — One-Time Setup

Run these once, in order, in a Snowsight worksheet:

- 01_stages_and_formats.sql — creates the SALES_ANALYTICS database, Bronze/Silver/Gold schemas, warehouse, JSON file format, and the internal RAW_LANDING stage.
- 01b_put_sample_data.sql — via SnowSQL (snowsql -f 01b_put_sample_data.sql), run from the sql/ folder so the relative sample_data/ paths resolve. No SnowSQL? Use Snowsight: open the BRONZE.RAW_LANDING stage, "+ Files", and upload each day's 4 files into a matching lead_activites_raw/dt=2026-08-03/ style subfolder -- tedious across 7 days by hand, so installing SnowSQL for this one step is worth it.
- 02_bronze_ddl.sql — creates the four Bronze tables.
- 04_json_repair_udf.sql — creates the two repair UDFs and runs the smoke test at the bottom; confirm the repaired_text output matches the expected string in the comment before moving on.
- 05_silver_transient_flatten.sql — creates every Silver table plus the transient-load procedure.
- 06_silver_merge_dedup.sql — creates the MERGE_SILVER_DAY procedure (this just defines it; nothing runs yet).
- 07_gold_views.sql — creates the five Gold views.
- 08_export_unload.sql — creates the four report views (the COPY INTO statements at the bottom will run immediately against whatever's in Silver so far -- that's fine, just re-run after Step 5 for real numbers).

5. Step 3 — Simulate the 7-Day Daily Run

This is the heart of the "runs daily for 7 days" requirement. For each simulated day, call the three load procedures in order, then spot-check row counts. Run this block seven times, changing the date each time (matching what 01b_put_sample_data.sql pushed to the stage):

```
CALL BRONZE.LOAD_BRONZE_DAY('2026-08-03');
CALL SILVER.LOAD_SILVER_TRANSIENT('2026-08-03');
CALL SILVER.MERGE_SILVER_DAY('2026-08-03');

SELECT COUNT(*) AS activities_in_silver FROM SILVER.LEAD_ACTIVITIES_PROCESSED;
SELECT COUNT(*) AS activities_in_summary FROM
SILVER.LEADS_ACTIVITIES_SUMMARY;
```

Then repeat for 2026-08-04 through 2026-08-09 (7 calls total). Expected activity counts in Silver after each day:

Day	Date	Cumulative activities expected

1	2026-08-03	9
2	2026-08-04	~14
3	2026-08-05	~19
4	2026-08-06	~23 (Lead C's correction lands here)
5	2026-08-07	~26
6	2026-08-08	~28
7	2026-08-09	30

Exact numbers will vary slightly since the filler leads (H-O) use randomized day offsets, but the count should only ever grow or stay flat -- never drop, and never double-count the same activity twice. To confirm the dedup logic (not just append) is actually working, run this after day 4 and again after day 7 -- the row count for this one activity should stay at exactly 1, but its outcome text should flip:

```
SELECT ACTIVITY_ID, TEXT, DATE_UPDATED, MD5_HASH
FROM SILVER.LEAD_ACTIVITIES_PROCESSED
WHERE LEAD_ID = 'lead_C' AND CUSTOM_ACTIVITY_TYPE_ID = '300';
```

6. Step 4 — Verify Gold and Run Export

After all 7 days are merged, spot-check the Gold views against the worked examples:

```
SELECT * FROM GOLD.SALES_DETAILS ORDER BY LEAD_ID;
-- expect lead_A ($15,000) and lead_D ($12,000)

SELECT LEAD_ID, STATUS, STRATEGY_CALL_OUTCOME, OFFER_PRESENTED
FROM GOLD.ALL_STRATEGIES_DETAILS ORDER BY LEAD_ID;
-- expect lead_C and lead_F: STATUS='Show', but no row for either in SALES_DETAILS

SELECT LEAD_ID, STRATEGY_CALL_BOOKED FROM
GOLD.OUTBOUND_STRATEGIES_BOOKED WHERE LEAD_ID = 'lead_G';
-- expect no row at all -- Lead G never got a Strategy Call scheduled

SELECT LEAD_ID, WRAPPED_ACTIVITIES:"Objections Faced" AS OBJECTIONS
FROM SILVER.LEADS_ACTIVITIES_SUMMARY WHERE LEAD_ID = 'lead_F' AND
CUSTOM_ACTIVITY_TYPE_ID = '300';
-- expect a 2-element ARRAY: ["Wasn't Looking For What We Offered", "Money"] -- apostrophe
intact,
-- this is the JSON-repair UDF's main real-world test case (Phase 3 Sec. 8)

SELECT * FROM GOLD.RPT_OBJECTIONS_FACED WHERE CLOSER_NAME LIKE
'%Whitfield%';
-- expect lead_F's call to increment BOTH not_looking_count and money_count (multi-select
fan-out)
```

Then re-run the four unloads in 08_export_unload.sql (or just call the wrapper once 09 is loaded):

```
CALL GOLD.EXPORT_ALL_REPORTS();
LIST @EXPORT_STAGE_OBJECTS.REPORT_EXPORTS;
```

Download the four CSVs from Snowsight (click into the stage, select a file, Download) and open them -- column headers should match the exact lists in Phase 1 Section 7 / Phase 2 Section 6.

7. Step 5 — Orchestration (Optional for the Trial Run)

09_orchestration_tasks_streams.sql creates the Streams and the Task DAG, but leaves every task SUSPENDED -- on a trial account, running the manual CALL sequence in Section 5 gives you full control over which "day" you're simulating, which the real CRON schedule doesn't. Resume the DAG (leaf-to-root, as shown in the file's comment block) once you're ready to let it run unattended against live daily source data.

8. Deviations From Phase 1 / Phase 2 (Read Before SME Review)

Implementing the actual SQL surfaced a few places where the design needed a small, documented adjustment:

- MAJOR REVISION -- custom activity field resolution: the original build modeled CUSTOM_ACTIVITIES as a flat outcome dictionary and assumed custom fields arrived as one stringified blob per activity (custom_fields_raw). After reviewing the project's explanation video and GoodNotes transcript, this was wrong on both counts. The real shape: each custom field on an activity is its own top-level custom.cf_<hash> key (an opaque, Close-generated field ID), and custom_activites_raw delivers a two-level dictionary -- one CustomActivityType per activity type, each with a nested fields[] array mapping field ID to human name. Fixed by replacing CUSTOM_ACTIVITIES with CUSTOM_ACTIVITY_TYPES + CUSTOM_ACTIVITY_FIELDS (05_silver_transient_flatten.sql), and rewriting the WRAPPED_ACTIVITIES build in 06_silver_merge_dedup.sql to FLATTEN every custom.cf_<hash> key, resolve it per-field via CUSTOM_ACTIVITY_FIELDS, repair only the values that are single-quoted pseudo-JSON (multi-select fields like Objections Faced -- not every field, as originally assumed), and OBJECT_AGG the result back together. CUSTOM_ACTIVITY_ID is renamed CUSTOM_ACTIVITY_TYPE_ID everywhere, and the old CUSTOM_ACTIVITY_OUTCOME_ID addition is dropped -- it's no longer needed now that outcome text is read directly out of WRAPPED_ACTIVITIES per activity type (Phase 2 Sec. 4.1/7 updated to match).
- Setter/Closer resolution in Gold (07_gold_views.sql) now reads primarily from that activity's own WRAPPED_ACTIVITIES:"Setter"/"Closer" custom fields (confirmed via the video that Close captures these directly on the Strategy Call / New Sale activity), joined to CLOSE_CRM_USERS_PROCESSED, falling back to the original look-back-to-booking-view logic only when that field is blank.
- Objections Faced is a genuine multi-select field -- a Lost call can carry more than one objection (e.g. Lead F: ["Wasn't Looking For What We Offered", "Money"]). GOLD.RPT_OBJECTIONS_FACED (08_export_unload.sql) now LATERAL FLATTENs the resolved array so a multi-objection call correctly increments every matching category, instead of only ever matching one exact string as a scalar comparison would.
- Bronze grain corrected from "1 row per activity record" to "1 row per file load" (03_bronze_copy_into.sql header comment) -- COPY INTO on a {"data": [...]} document lands the whole payload as one VARIANT row; per-activity granularity only exists from the Silver FLATTEN step onward.

- Resolved the Phase 2 ACTIVITY_ID-vs-LEAD_ID+ACTIVITY_ID open item: ACTIVITY_ID is the MERGE key (treated as globally unique, consistent with Close CRM), with LEAD_ID + ACTIVITY_ID used only for within-batch dedup via QUALIFY ROW_NUMBER() as a defensive measure.
- PROGRAM in SALES_DETAILS is read from WRAPPED_ACTIVITIES:"Program" -- the sample generator populates this on New Sale activities, but confirm the real source field name with the SME before Phase 4 / production cutover.
- The Objections Faced Report is scoped to Strategy Call activities with outcome '7. Lost' (08_export_unload.sql) -- objections are only meaningful for calls that didn't close. Confirm this matches SME intent; if any non-Lost outcome can also carry an objection, the WHERE clause needs loosening.
- STATUS in OUTBOUND_PROSPECT_DIALS is aliased directly from CUSTOM_ACTIVITY_OUTCOME_NAME as a simplification -- flag if a distinct dial-disposition status is expected instead.

9. Moving to Production

- Swap the internal RAW_LANDING stage for a real S3 external stage -- the commented-out block at the bottom of 01_stages_and_formats.sql shows the exact STORAGE INTEGRATION + CREATE STAGE swap. No other script changes needed; everything downstream reads the stage by name.
- Point the daily extract job at the real PostgreSQL source (connection details in the original requirements doc) instead of the sample data generator.
- Resume the Task DAG (Section 7) so BRONZE.TASK_LOAD_BRONZE picks up the real daily 07:30 AM EST refresh on its own schedule.
- Re-run the Gold/Export spot-checks in Section 6 against real data before sharing report output with stakeholders, per the original requirement to "ping Avirup for the actual report result and verify using date based output."